

PROBLEM STATEMENT

1. Context & Background

Academic institutions rely heavily on physical library resources to support student learning, research, and coursework execution. However, traditional library operations often depend on manual checkout procedures, physical bulletin boards, or desktop-only web portals. With the widespread adoption of mobile technology among higher education students, there is a growing expectation for immediate, on-the-go access to academic services.

2. The Problem

Current library resource management methods suffer from severe operational inefficiencies and poor user visibility:

- **Lack of Real-Time Inventory Visibility:** Students often travel to physical library locations only to find that required textbooks are out of stock or reserved by others.
- **Inefficient Rental Tracking & Overdue Penalties:** Without automated mobile tracking, students struggle to keep track of return deadlines, leading to unexpected overdue fines and unreturned library assets.
- **Friction in Reservation Lifecycles:** Processing renewals, returns, or reservation cancellations manually causes administrative bottlenecks for library staff and creates unnecessary delays for waiting students.
- **Lack of Local Off-Line Availability:** Web-based library catalogue tools often perform poorly on mobile networks and fail to retain offline session states.

3. Impact of the Problem

These limitations result in decreased student engagement, underutilized library inventories, high administrative overhead for institutions, and frustration among students who cannot efficiently secure course literature critical to their academic success.

Note: Construct your Mobile App Design & Development based on the following instruction.

TASK	ASSIGNMENT INSTRUCTION (120 MARKS)
1	Assignment: SECTION A: UI LAYOUT & NAVIGATION DESIGN (40 MARKS) 1. Book Catalog Screen (15 Marks) • Catalog Layout: Build a responsive grid or vertical list (LazyColumn or LazyVerticalGrid) showcasing available and borrowed books. • Item Card & Badging: Each card must visually indicate title, author, category, and availability status (e.g., green badge for "Available", red/orange badge for "Borrowed"). • Search & Filter: Implement a top bar search widget (OutlinedTextField or SearchBar) that allows users to filter the list dynamically by book title or author. • Reservation Action: Tapping an available book triggers a Modal Bottom Sheet or Dialog containing rental duration choices and a "Reserve" button. 2. User Reservations Dashboard (15 Marks) • Active Bookings View: A dedicated dashboard screen featuring a LazyColumn of active rentals. • Information Display: Each item card must display book details, reservation date, return deadline, and calculated days remaining. • Lifecycle Action Controls: ○ Renew Button: Allows updating/extending the return deadline.

- **Return Button:** Processes the return action and frees up book availability.
- **Cancel Button:** Cancels pending reservations prior to pickup.

3. Navigation & Reactive State Integration (10 Marks)

- Implement single-activity navigation using Jetpack Navigation Compose (NavHost, composable, NavController).
- Utilize state hoisting and observe screen state via collectAsStateWithLifecycle() or collectAsState(). The catalog status badge must immediately reflect state changes when a book is reserved or returned.

SECTION B: ROOM DATABASE & BUSINESS LOGIC (40 MARKS)

1. Entities (12 Marks)

Define two related Room entities: BookEntity and BookingEntity.

- **BookEntity:**
 - bookId (Auto-generated Primary Key)
 - title (String)
 - author (String)
 - category (String)
 - isAvailable (Boolean — Default: true)
- **BookingEntity:**
 - bookingId (Auto-generated Primary Key)
 - bookOwnerId (Foreign Key referencing BookEntity.bookId with CASCADE delete)
 - userName (String)
 - bookingDate (Long / Timestamp)
 - returnDeadline (Long / Timestamp)
 - status (Enum: PENDING, ACTIVE, RETURNED)

2. Data Access Object (DAO) (18 Marks)

Create BookDao and BookingDao (or a combined LibraryDao) supporting full CRUD operations for the rental lifecycle:

- **Create (C):** Insert new books and record a new BookingEntity upon reservation.
- **Read (R):**
 - Fetch books filtered by availability as a Flow<List<BookEntity>>.
 - Fetch active reservations joined with relevant book info for the dashboard view.
 - Search books using SQL LIKE queries (%query%).
- **Update (U):**
 - Extend rental periods (update returnDeadline on BookingEntity).
 - Process returns (update BookingEntity status and switch BookEntity.isAvailable back to true).
- **Delete (D):**
 - Cancel pending bookings (delete the BookingEntity transaction row before pickup).

3. Database Class & Converters (10 Marks)

- Define @Database(entities = [BookEntity::class, BookingEntity::class], version = 1).
- Implement a @TypeConverters class for complex data mapping (e.g., converting Enums or Date / Timestamp to SQLite primitives).
- Provide a thread-safe Singleton database instance using Room.databaseBuilder, configured to pre-populate sample book records on initial setup.

SECTION C: ARCHITECTURE & MVVM SETUP (15 MARKS)

1. MVVM Layering (10 Marks):

	○ Maintain clean separation of concerns: UI Layer (Compose Screens), ViewModel (State Management), Repository (Business logic abstraction), and Data Layer (DAOs & Entities). 2. Gradle Configuration & Dependencies (5 Marks): ○ Proper setup of KSP/KAPT annotation processing for Room, Navigation Compose, and ViewModel Compose extensions. SECTION D: ADVANCED FEATURES & CODE QUALITY (15 MARKS) 1. Reactive Database Observing (5 Marks): ○ UI elements must reactively observe database changes via Kotlin Flow emitted directly from Room DAOs. 2. Asynchronous Execution (5 Marks): ○ Ensure zero database transactions run on the Main Thread. All calls must be executed asynchronously using Kotlin Coroutines (viewModelScope.launch). 3. Clean Code & Input Validation (5 Marks): ○ Sanitize text fields, prevent invalid return dates, follow Kotlin style guidelines, and organize source code into logical packages (ui/, data/, di/, domain/).
	Monitor Day 1: 28/08/2026 [5 marks] • Mobile App Design Layout & progress presented to the lecturer. (3 marks) • At least 50% done (2 marks)
	Monitor Day 2: 11/09/2026 [5 marks] • Mobile App Database & Progress presented to the lecturer. (3 marks) • At least 50% done (2 marks)

ADDITIONAL INFORMATION TO NOTE	
	Note: Things to keep in mind when working with Room, Kotlin, and Jetpack Compose: 1. Threading & Concurrency • No Main-Thread Operations: Room crashes if executed on the main UI thread. Always use Kotlin Coroutines (suspend functions) or Flow for database access. • Asynchronous Execution: Run write operations (Insert/Update/Delete) inside viewModelScope.launch { ... }. 2. Dependency Setup • Use KSP over KAPT: Use the Kotlin Symbol Processing (ksp) plugin in build.gradle.kts instead of kAPT for significantly faster compile-time code generation. 3. Jetpack Compose Integration • Lifecycle-Aware Collection: In Compose screens, collect state using collectAsStateWithLifecycle() to automatically pause stream collection when the UI is in the background. • Expose via StateFlow: Convert Room Flow queries to StateFlow in your ViewModel using .stateIn(scope, SharingStarted.WhileSubscribed(5000), initialValue). 4. Data Types & Entities • Type Converters: SQLite only handles basic primitives. Use @TypeConverters for complex types like Date, Instant, or Enums.

- Schema Changes: Increment @Database(version = N) whenever you modify an @Entity. Use .fallbackToDestructiveMigration() during initial development to prevent app crashes when updating schemas.

Project Specification Document (Guide)

Mobile Application Development: Library Book Rental & Reservation System

1. Project Overview

The Library Book Rental & Reservation System is a native Android application developed using Kotlin, Jetpack Compose, and Room Database. It enables users to browse books, reserve titles, manage rentals (renew, return, or cancel), and track return dates through a responsive interface.

2. Objectives (SMART)

- **Specific:** Develop a single-activity Android app with full CRUD functionality for book rentals.
- **Measurable:** Achieve the project requirements and assessment criteria (110 marks).
- **Achievable:** Use Kotlin, Jetpack Compose, Room, MVVM, Coroutines, and Flow.
- **Relevant:** Apply third-year mobile development principles, including local data persistence and reactive UI.
- **Time-Bound:** Complete all milestones and submit the final project by the end of the term.

3. Scope

In Scope

- Jetpack Compose (Material 3) user interface.
- Room Database (BookEntity and BookingEntity).
- Coroutines and Flow for asynchronous data handling.
- Book search, filtering, reservations, renewals, returns, and cancellations.
- Single-activity navigation using Navigation Compose.

Out of Scope

- Firebase, REST APIs, or cloud databases.
- User authentication or multi-user support.
- Payment processing for late fees.

4. Deliverables

- Complete Kotlin project in a public GitHub repository.
- Pre-populated Room database with sample books.
- README.md containing setup instructions, architecture overview, and CRUD screenshots/GIFs.
- Progress demonstrations on Monitor Days.

5. Timeline & Milestones

Milestone	Date	Focus	Deliverable
Monitor Day 1	28/08/2026	UI Design	Minimum 50% of Compose UI and navigation completed (5 Marks)
Monitor Day 2	11/09/2026	Database Integration	Minimum 50% of Room DB, DAO, and ViewModel completed (5 Marks)
Final Submission	End of Term	Complete Project	GitHub repository and README.md (110 Marks)

6. Budget

- **Cost:** R0 (Uses free, open-source software and libraries.)

7. Resources

- Software: Android Studio (Ladybug or newer), JDK 17+, Git, GitHub.
- Technologies: Kotlin, Jetpack Compose, Room, KSP, Coroutines, StateFlow, Navigation Compose.
- Hardware: Android emulator or Android device (API 26+).

8. Assumptions & Constraints

Assumptions

- Students understand **Kotlin** fundamentals, **@Composable** and **object-oriented programming**.

Constraints

- No database operations on the Main Thread.
- All interfaces must be built using **Jetpack Compose** (XML layouts are **not** permitted).

9. Risk Management

Risk	Severity	Mitigation
Main Thread database access	High	Use Coroutines within viewModelScope and observe data with Flow.
Database migration issues	Medium	Use fallbackToDestructiveMigration() during development.
Room/KSP build errors	Medium	Keep Kotlin, Room, and KSP versions compatible.

10. Quality Standards

- Follow MVVM architecture.
- Write clean, modular, and well-documented Kotlin code.
- Deliver a responsive, stable, and user-friendly interface.

11. Communication

- Progress reviews during Monitor Days.
- Support through consultations, labs, and discussion forums.
- Submit the **GitHub** repository link via project document.

12. Acceptance Criteria

The application must:

- Run successfully on Android API 26+ without crashes.
- Support browsing, searching, and reserving books.
- Allow users to renew, return, and cancel reservations.
- Update the UI reactively using Room and Kotlin Flow.
- Include a complete GitHub repository with a well-documented **README.md**.

FORMAT

Project files to be uploaded to Canvas as a single **Zip File** with your *StudName_MADB372_SF1.docx*:

- A. The Android Mobile App file/Folder is named **Kitabu_MADB372_SF1**.
- B. In the **Zip** file also includes the **App Screenshots**: saved in a **Word document** *StudName_MADB372_SF1.docx*

TECHNICAL REQUIREMENTS

To successfully complete this assignment, you will need to make use of the following tools:

- **Android Studio**
- **Figma** (*for Design/Prototyping*)
- **Room Database**
- **Microsoft Word/pdf**
- **Canvas**

CHECKLIST

Once you have completed your assessment, use the checklist below to ensure that you have covered all elements.

	Task/ Requirement	YES	NO
1	Did you upload all project files in a neatly structured and organized and named appropriately. Ensure that you submit all deliverables including project: - Any project resources - Documentation - Visual design Plans.		
2	File Naming: Follows the specified format with all assignment files		
3	Evidence of AI Use: If applicable, include screenshots of prompts and responses. <i>Ensure you include all your planning documents. If AI is used, include screenshots of your prompts and responses. This evidence will ensure that your assessment is valid and authentic. Without this evidence, your assessment will not be marked.</i>		
4	Technical Tools Used: Android Studio, MS Word...etc		
5	Font and Spacing: Use Calibri, 10-point, 1.5 line spacing for documents.		
6	Did you include a list of references / in-text referencing?		



Useful Resource:

- Kotlin: [Kotlin Official Documentation](#)
- Jetpack Compose: [Android Developers - Jetpack Compose Documentation](#)
- Room Database: [Android Developers - Room Persistence Library](#)
- GitHub: [GitHub Official Documentation & Guides](#)
- SQL (SQLite for Android): [SQLite Official Documentation](#) & [Android SQLite Query Reference](#)
- Figma: [Figma Help Center & Official Documentation](#)
- Android Studio: [Android Studio User Guide & Tools Documentation](#)